

VINUNIVERSITY
COLLEGE OF ENGINEERING AND COMPUTER SCIENCE



Mid-term Project Report
Project Name: MediTrack - Hospital
Management System

COURSE CODE: ELEC2030

Supervisor	: Dr. Do Danh Cuong	
Student 1	: Hoang Manh Kien	Student ID: V202300984
Student 2	: Nguyen Huy Long	Student ID: V202401304
Student 3	: Dinh Nguyen Phuong	Student ID: V202300991

HANOI – Spring, 2025



1. INTRODUCTION:

1.1 Member List & Workload

No.	Full Name	Student ID	Responsibility	Contribution
1	Hoang Manh Kien	V202400984	- Algorithm, Data Structure - Testing, Error Handling - Video Editor - Pseudocode	100%
2	Nguyen Huy Long	V202401304	- Algorithm, Data Structure - Flowchart and Module - Pseudocode	100%
3	Dinh Nguyen Phuong	V202300991	- Product Overview - Problem - Conclusion & Future Work	100%

1.2 Overview of Product:

MediTrack is a **console-based desktop program** developed in the **C programming language**. It is not a mobile app or web-based application but rather a command-line utility focused on hospital information management. The program operates through a text-based interface and stores its data locally using file I/O, making it suitable for low-resource environments or offline systems. This implementation is especially useful in academic contexts to demonstrate mastery of data structures (like Linked Lists, Queues, and Binary Search Trees) and file operations in C.

The primary target users of **MediTrack** are:

- **Small clinics or local medical centers** that require basic digital patient management but may not have access to advanced hospital software.
- **University students or educators** who are learning about system design, C programming, and data structures.



- **Developers or interns** prototyping hospital systems before scaling to full GUI or web apps.

To operate, we have these modules and functions:

Module: patient.c (Patient Management):

- **add_patient()**: Adds a new patient to the linked list by taking user input for name, age, and gender. A unique patient ID is generated and assigned. The patient is added at the end of the list.
- **search_patient()**: Searches the linked list for a patient by name. If found, it displays the patient's information; otherwise, it notifies the user that no match was found.
- **display_patient_info()**: Traverses the entire patient linked list and prints each patient's ID, name, age, and gender in a readable format.
- **delete_patient()**: Removes a patient node from the linked list by name. It adjusts the pointers to maintain the list structure and frees the memory.
- **save_patient_to_file()**: Writes the current list of patients to patients.txt so that the data can be reloaded in the future.
- **load_patient_from_file()**: Loads patient data from patients.txt into the linked list during program startup.
- **free_list()**: Frees the dynamically allocated memory used by the patient list to avoid memory leaks on program exit.

Module: doctor.c (Doctor Management)

- **init_doctor(Vector* v)**: Initializes a dynamic array (vector) to store doctor records, setting an initial capacity and size.
- **add_doctor(Vector* v)**: Adds a new doctor to the dynamic array. Automatically expands the array capacity if needed.
- **list_doctor(Vector* v)**: Displays all stored doctor names and their corresponding IDs to the user.
- **delete_doctor_by_id(Vector* v)**: Removes a doctor from the array based on a matching name. Shifts the remaining doctors to maintain array continuity.
- **save_doctor_to_file(Vector* v)**: Saves the current list of doctors to doctors.txt for persistent storage.
- **load_doctor_from_file(Vector* v)**: Reads doctor data from doctors.txt and loads it into the dynamic array at startup.

Module: appointment.c (Appointment Scheduling)



- `init_appointment_system(AppointmentSystem* system)`: Initializes the appointment system by setting the BST root and queue pointers to NULL, and resetting the appointment count.
- `book_appointment()`: Books a new appointment by inserting it into both a BST (for time-sorted access) and a queue (for FIFO processing). Rejects appointments that conflict in time.
- `get_next_appointment()`: Returns the appointment at the front of the queue — the one that was booked earliest.
- `find_appointments_in_range()`: Retrieves all appointments scheduled between two time values using a recursive range query on the BST.
- `display_appointment(const Appointment* appt)`: Prints the details of a single appointment, including patient ID, doctor ID, time, and purpose.
- `free_appointment_system()`: Frees all memory allocated for both the BST and the queue used in the appointment system.

Module: utils.c (Utilities)

- `generate_id(const char* entity)`: Generates a unique, incremental ID for patients, doctors, or appointments. The last used ID is read from and updated in a file (data/patient_id.txt, etc.), ensuring uniqueness across sessions.

2. PROBLEM:

While **MediTrack** fulfills core hospital management tasks such as patient handling, doctor listing, and appointment scheduling, we acknowledge several current limitations that we aim to improve in future versions:

1. **Incomplete Date-Time Handling:** The appointment system currently only manages time within a 24-hour format (e.g., 09:00, 14:30), and does not support multiple days, months, or years. This limits the system to a single-day schedule. Future versions will expand the datetime structure to include full calendar support for long-term appointment planning.
2. **Appointment Conflict Detection:** The system blocks identical appointment times but does not yet handle overlapping bookings. We aim to add logic to prevent appointments from being scheduled too close together or during an existing doctor's booked time slot.

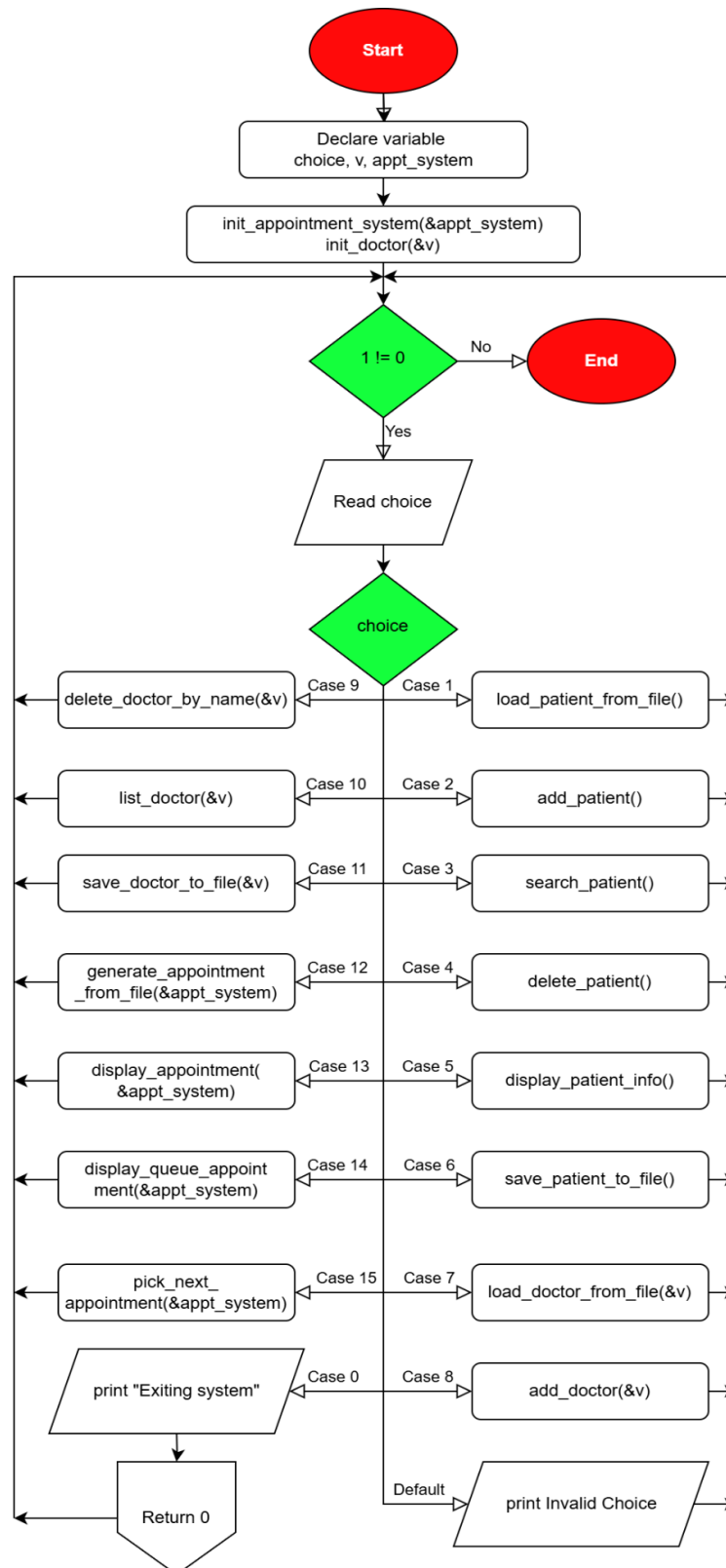


3. **Patient and Doctor Validation:** When booking an appointment, the system accepts any entered patient or doctor ID without verifying their existence. A validation step will be introduced to check ID validity before confirming appointments.
4. **Unencrypted Data Storage:** All information is stored in plaintext .txt files, which may pose privacy concerns for real-world usage. We plan to upgrade to a more secure format or migrate to a simple local database (e.g., SQLite) in the future.
5. **Basic Search Capability:** Patient search currently supports only exact name matching. Enhancing this with partial search, fuzzy matching, or filters (age, gender) will improve usability, especially in large datasets.
6. **Command-Line Interface (CLI) Limitations:** The current CLI is functional but not user-friendly for staff unfamiliar with technical systems. We plan to introduce a graphical or menu-based UI to make the system more intuitive.



3. SOLUTION:

3.1 Overall Flowchart



4

Figure 1: Flow Chart of Main.c

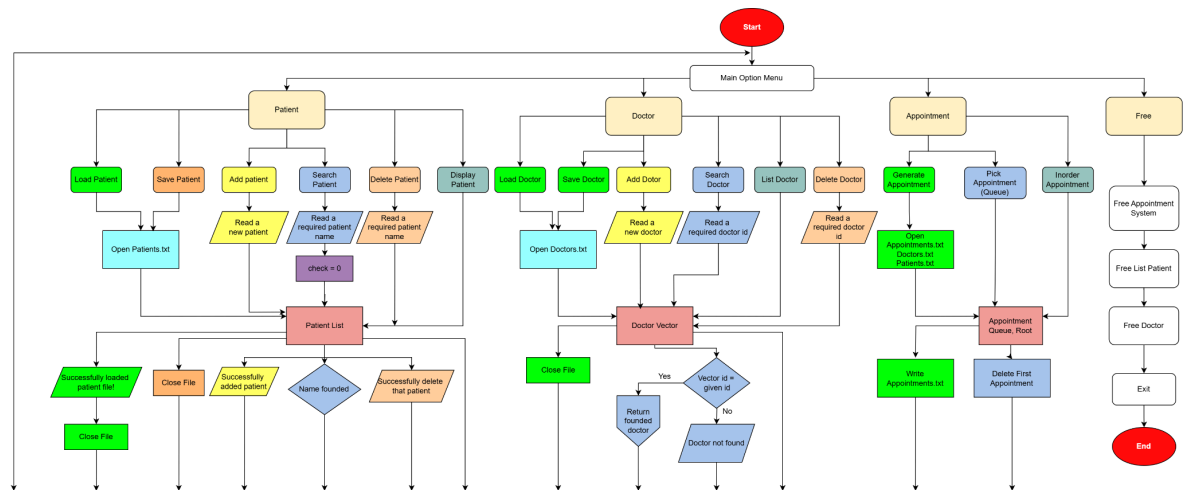


Figure 2: Full Flowchart of Program

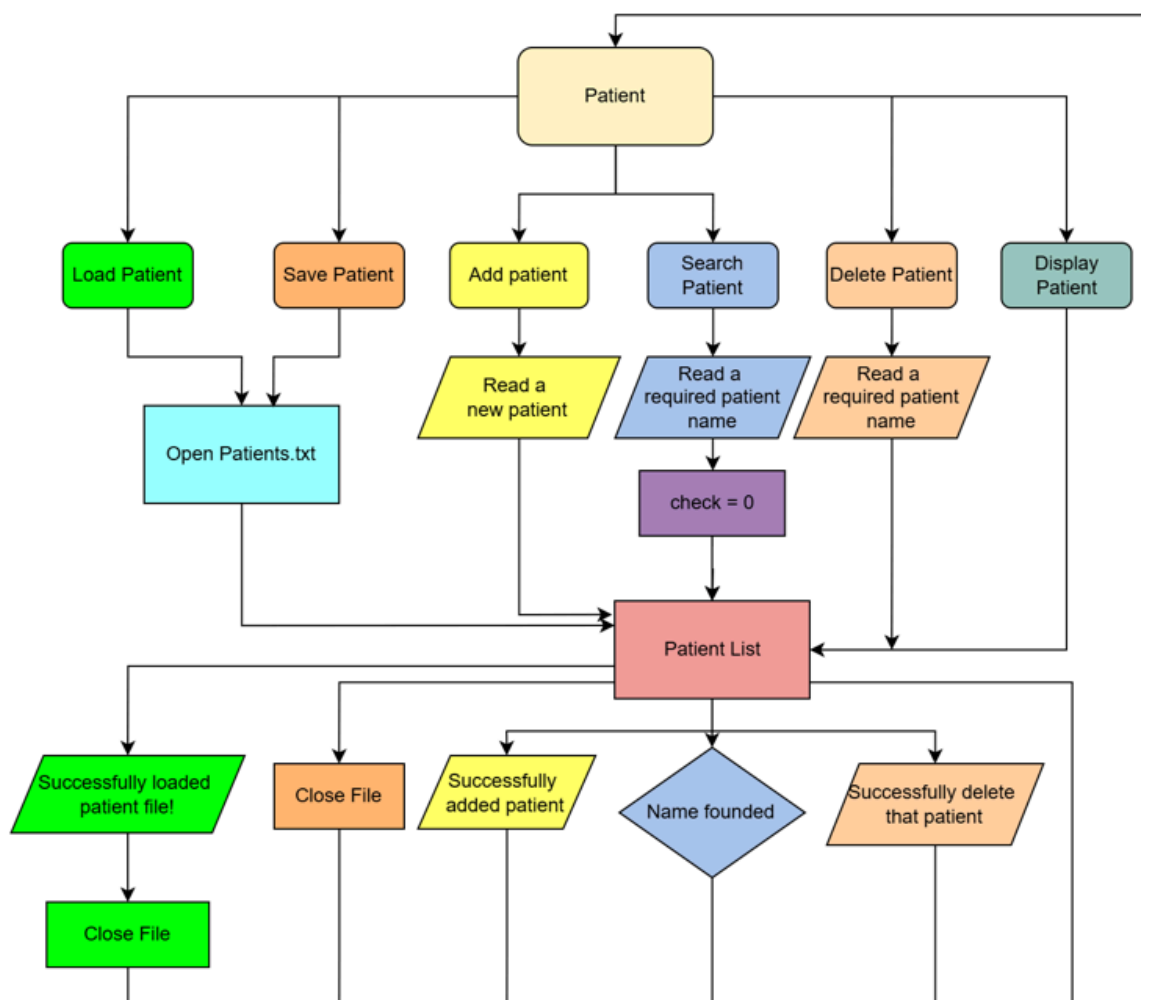


Figure 3: Module Patient

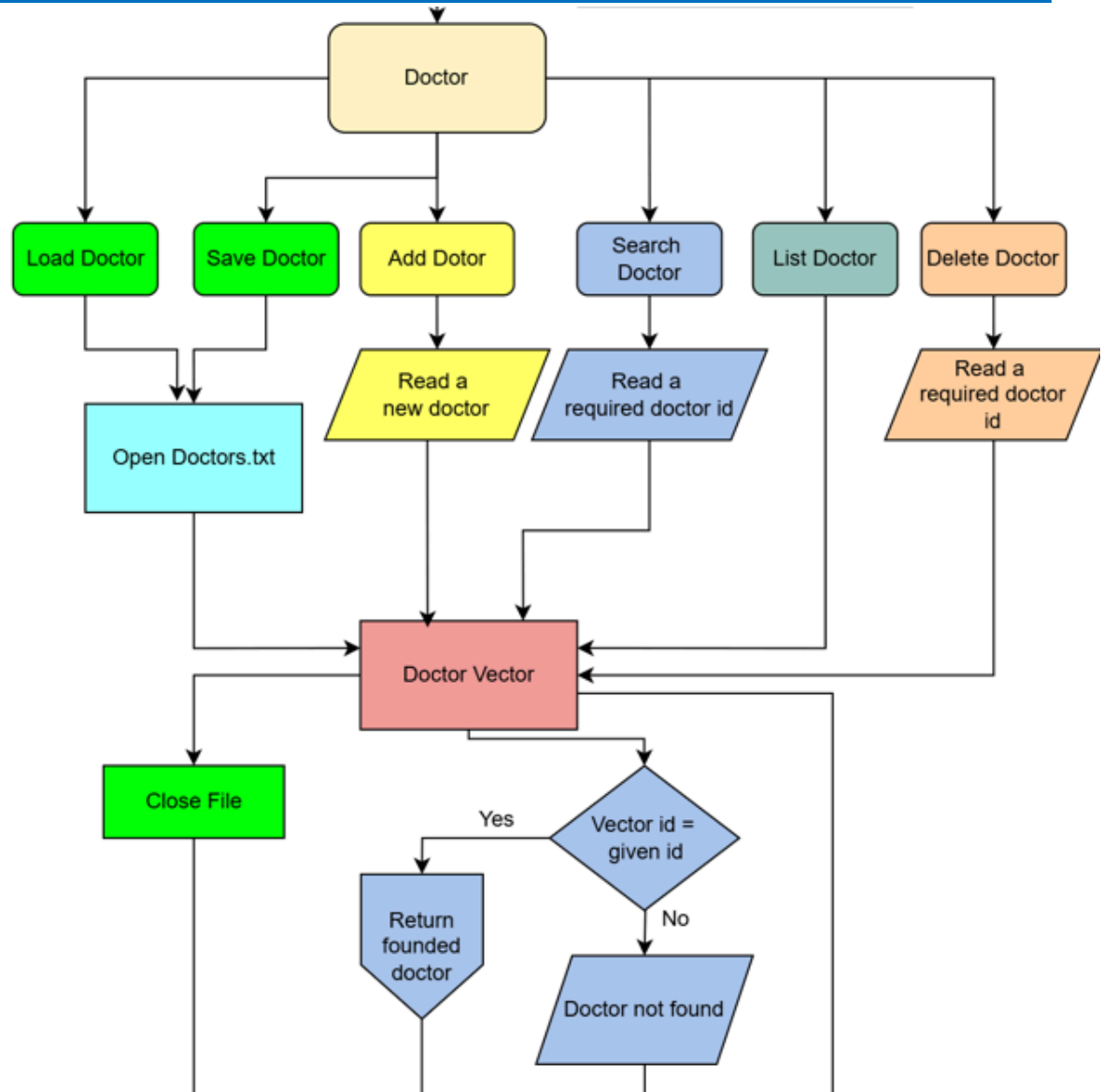


Figure 4: Module Doctor

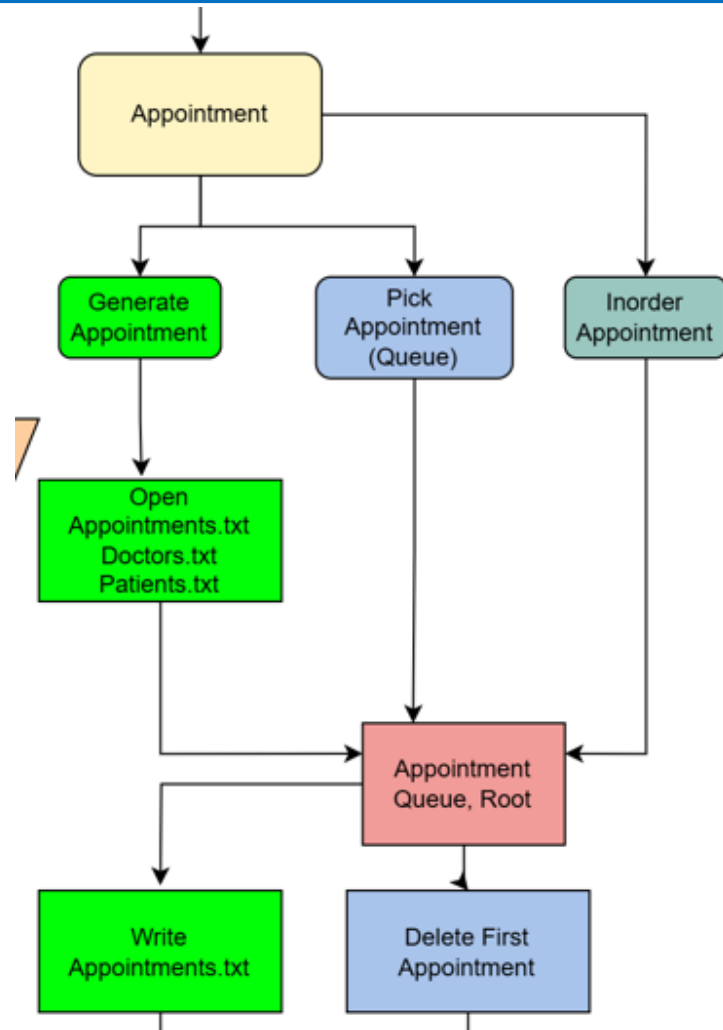


Figure 5: Module Appointment

3.2 Modules architecture

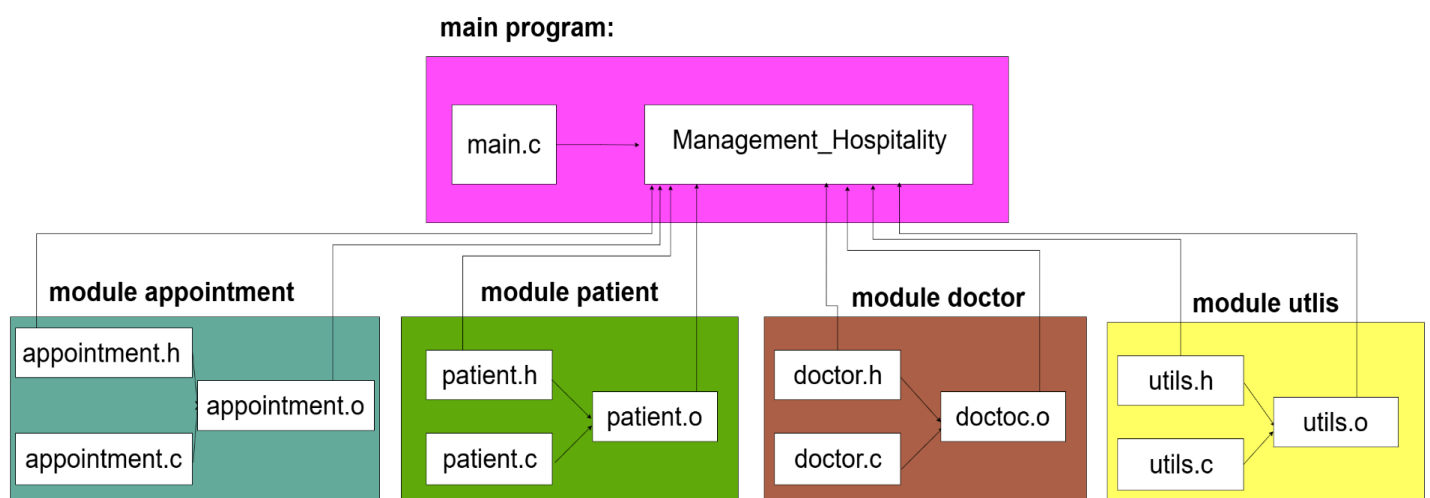


Figure 6: Module Management Hospitality Project

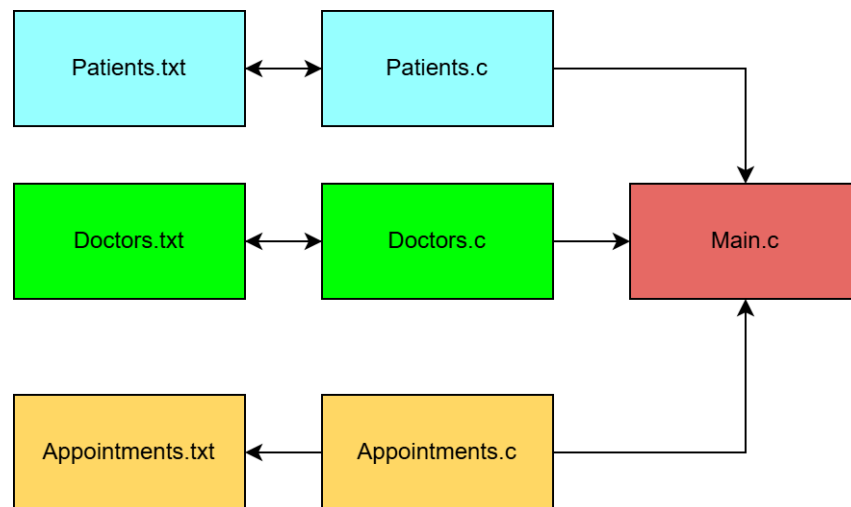


Figure 7: File Input / Output Management

3.3 Algorithms of all important functions

DOCTOR.C FILE FUNCTIONS

BST Insertion Algorithm

Algorithm INSERT_DOCTOR(root, doctor):

```
IF root == NULL THEN
    newNode = ALLOCATE_MEMORY(sizeof(Node_doctor))
    newNode->doctor = doctor
    newNode->left = NULL
    newNode->right = NULL
    RETURN newNode
IF doctor.id > root->doctor.id THEN
    root->right = INSERT_DOCTOR(root->right, doctor)
ELSE IF doctor.id < root->doctor.id THEN
    root->left = INSERT_DOCTOR(root->left, doctor)
RETURN root
```

BST Search Algorithm

Algorithm SEARCH_DOCTOR(root, doctor_id):

```
IF root == NULL THEN
    PRINT "Doctor not found"
```



EXIT PROGRAM

```
IF root->doctor.id == doctor_id THEN
    RETURN root->doctor
ELSE IF doctor_id > root->doctor.id THEN
    RETURN SEARCH_DOCTOR(root->right, doctor_id)
ELSE
    RETURN SEARCH_DOCTOR(root->left, doctor_id)
```

Inorder Traversal Algorithm

Algorithm INORDER_TRAVERSAL(root):

```
IF root != NULL THEN
    INORDER_TRAVERSAL(root->left)
    PRINT "Name:", root->doctor.name, "ID:", root->doctor.id
    INORDER_TRAVERSAL(root->right)
```

Find Minimum Node Algorithm

Algorithm FIND_MIN_NODE(root):

```
WHILE root->left != NULL DO
    root = root->left
RETURN root
```

BST Deletion Algorithm

Algorithm DELETE_DOCTOR(root, doctor_id):

```
IF root == NULL THEN RETURN NULL
IF doctor_id < root->doctor.id THEN
    root->left = DELETE_DOCTOR(root->left, doctor_id)
ELSE IF doctor_id > root->doctor.id THEN
```



```
    root->right = DELETE_DOCTOR(root->right, doctor_id)
ELSE
    // Node found
    IF root->left == NULL THEN
        temp = root->right
        FREE root
        RETURN temp
    ELSE IF root->right == NULL THEN
        temp = root->left
        FREE root
        RETURN temp
    ELSE
        temp = FIND_MIN_NODE(root->right)
        root->doctor = temp->doctor
        root->right = DELETE_DOCTOR(root->right, temp->doctor.id)
    RETURN root
```

Save Doctors to File Algorithm

```
Algorithm SAVE_DOCTORS_TO_FILE(v):
    file = OPEN_FILE("data/doctors.txt", WRITE_MODE)
    IF file == NULL THEN RETURN
    FOR i = 0 TO v.size-1 DO
        WRITE_TO_FILE(file, v.data[i].name, v.data[i].id)
    CLOSE_FILE(file)
```

Delete Doctor by Name Algorithm

```
Algorithm DELETE_DOCTOR_BY_NAME(v):
    PRINT "Doctor Name List:"
    FOR i = 0 TO v.size-1 DO
        PRINT v.data[i].name
    name = GET_INPUT("Enter name to delete")
    found_index = -1
    // Find in vector
```



```
FOR i = 0 TO v.size-1 DO
    IF v.data[i].name == name THEN
        found_index = i
        BREAK
IF found_index == -1 THEN
    PRINT "Doctor not found"
    RETURN
root = DELETE_DOCTOR(root, v.data[found_index].id)
FOR i = found_index TO v.size-2 DO
    v.data[i] = v.data[i+1]
v.size -= 1
```

Add New Doctor Algorithm

Algorithm ADD_DOCTOR(v):

```
PRINT "Enter doctor name:"
name = GET_INPUT()
PRINT "Enter doctor ID:"
id = GET_INTEGER_INPUT()
IF v.size == v.capacity THEN
    v.capacity *= 2
    REALLOCATE_MEMORY(v.data, v.capacity * sizeof(Doctor))
doctor = CREATE_DOCTOR(name, id)
v.data[v.size] = doctor
v.size += 1
root = INSERT_DOCTOR(root, doctor)
```

PATIENT.C FILE FUNCTIONS

Add Patient Algorithm

Algorithm ADD_PATIENT():

```
// Input collection
new_patient = GET_PATIENT_INPUT()
// Node creation
```



```
new_node = ALLOCATE_MEMORY(sizeof(Node))
new_node->data = new_patient
new_node->next = NULL
// List insertion
IF head == NULL THEN
    head = new_node
ELSE
    current = head
    WHILE current->next != NULL DO
        current = current->next
    current->next = new_node
PRINT "Successfully added patient"
```

Search Patient Algorithm

Algorithm SEARCH_PATIENT():

```
PRINT "List of Patient Names:"
TRAVERSE_AND_PRINT_NAMES()
name = GET_INPUT("Enter patient name to search")
found = FALSE
current = head
WHILE current != NULL DO
    IF current->data.name == name THEN
        PRINT_PATIENT_INFO(current->data)
        found = TRUE
    current = current->next
IF NOT found THEN
    PRINT "Patient not found"
```

Display all Patients Algorithm

Algorithm DISPLAY_PATIENTS():

```
current = head
count = 0
WHILE current != NULL DO
```



```
count = count + 1
PRINT_FORMATTED(
    current->data.id,
    current->data.name,
    current->data.age,
    current->data.gender
)
current = current->next
PRINT "Total patients:", count
```

Delete Patient Algorithm

```
Algorithm DELETE_PATIENT():
    PRINT "List of Patient Names:"
    TRAVERSE_AND_PRINT_NAMES()
    name = GET_INPUT("Enter name to delete")
    current = head
    prev = NULL
    WHILE current != NULL DO
        IF current->data.name == name THEN
            IF prev == NULL THEN
                head = current->next
            ELSE
                prev->next = current->next
            FREE(current)
            PRINT "Successfully deleted"
            RETURN
        prev = current
        current = current->next
    PRINT "Patient not found"
```

APPOINTMENT.C FILE FUNCTIONS

Book Appointment

```
Algorithm BOOK_APPOINTMENT(system, patient, doctor, datetime):
```



```
appt.appointment_id = system.count + 1
appt.patient = patient
appt.doctor = doctor
appt.datetime = datetime
current = ADDRESS_OF(system.bst_root)
WHILE *current != NULL DO
    IF datetime < (*current).data.datetime THEN
        current = ADDRESS_OF((*current).left)
    ELSE IF datetime > (*current).data.datetime THEN
        current = ADDRESS_OF((*current).right)
    ELSE
        RETURN 0 // Time slot occupied
*current = NEW_BST_NODE(appt)
IF *current == NULL THEN RETURN 0
// Queue insertion
q_node = NEW_QUEUE_NODE(appt)
IF q_node == NULL THEN RETURN 0
IF system.queue_rear == NULL THEN
    system.queue_front = q_node
    system.queue_rear = q_node
ELSE
    system.queue_rear.next = q_node
    system.queue_rear = q_node
system.count++
RETURN 1
```

Generate Appointments from Files

Algorithm GENERATE_APPOINTMENTS_FROM_FILES(system):

```
OPEN doctor_file, patient_file, output_file
IF any file fails THEN
    CLOSE all opened files
    RETURN
current_time = GET_CURRENT_TIME()
```




```
year, month, day = EXTRACT_DATE(current_time)
hour = 0
minute = 0
WHILE READ_DOCTOR(doctor_file) AND READ_PATIENT(patient_file) DO
    appt_time = CREATE_TIME(year, month, day, hour, minute)
    IF BOOK_APPOINTMENT(system, patient, doctor, appt_time) THEN
        WRITE_TO_FILE(output_file, doctor, patient, appt_time)
        PRINT "Booked successfully"
    ELSE
        PRINT "Failed - time slot occupied"
    hour++
    IF hour >= 24 THEN
        hour = 0
        day++
WHILE READ_DOCTOR(doctor_file) DO
    PRINT "Unmatched Doctor:", doctor.name
WHILE READ_PATIENT(patient_file) DO
    PRINT "Unmatched Patient:", patient.name
CLOSE all files
```

Display Appointments (BST Inorder Traversal)

Algorithm DISPLAY_APPOINTMENTS(system):

```
IF system.bst_root == NULL THEN RETURN
INORDER_TRAVERSAL(system.bst_root)
```

Algorithm INORDER_TRAVERSAL(node):

```
IF node != NULL THEN
    INORDER_TRAVERSAL(node.left)
    PRINT_APPOINTMENT(node.data)
    INORDER_TRAVERSAL(node.right)
```

Algorithm PRINT_APPOINTMENT(appt):

```
PRINT "Appt #", appt.appointment_id
PRINT "Patient:", appt.patient.name, appt.patient.age, appt.patient.gender
PRINT "Doctor:", appt.doctor.name, appt.doctor.id
```



```
PRINT "Time:", FORMAT_TIME(appt.datetime)
```

Queue Operations

Algorithm DISPLAY_QUEUE(system):

```
current = system.queue_front  
IF current == NULL THEN  
    PRINT "No appointments in queue"  
    RETURN  
WHILE current != NULL DO  
    PRINT_APPOINTMENT(current.data)  
    current = current.next
```

Algorithm PICK_NEXT_APPOINTMENT(system):

```
IF system.queue_front == NULL THEN RETURN  
next = system.queue_front  
PRINT "Next Appointment:"  
PRINT_APPOINTMENT(next.data)  
system.queue_front = system.queue_front.next  
IF system.queue_front == NULL THEN  
    system.queue_rear = NULL  
FREE next
```



3.4 Implementation:

Module Overview

`doctor.c` & `doctor.h` – Doctor Management (Vector):

- Uses a dynamic array (Vector) to store doctors.
- Expands automatically when capacity is reached.

`patient.c` & `patient.h` – Patient Management

- Implements a singly linked list to manage patients dynamically.
- Includes full CRUD operations and file I/O for persistence.

`appointment.c` & `appointment.h` – Appointment System (BST + Queue)

- Manages appointments using:
 - A Binary Search Tree (BST) sorted by `datetime` for efficient searching
 - A Queue for processing appointments in booking order (FIFO)

`utils.c` & `utils.h` – ID Generation and Helpers

Code blocks

`patient.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "patient.h"
#include "utils.h"

Node* head = NULL;

void add_patient() {
    Patient new_patient;
```



```
printf("Enter the patient's name: ");
scanf("%49s", new_patient.name); // Limit input length

printf("Enter the patient's age: ");
while (scanf("%d", &new_patient.age) != 1 || new_patient.age <= 0) {
    printf("Invalid age. Please enter a positive number: ");
    while (getchar() != '\n'); // Clear input buffer
}

// Gender validation loop
do {
    printf("Enter the patient's gender (Male/Female): ");
    scanf("%9s", new_patient.gender); // Limit input length
    if (!is_valid_gender(new_patient.gender)) {
        printf("Invalid gender. Please enter either 'Male' or 'Female'.\n");
    }
} while (!is_valid_gender(new_patient.gender));

Node* new_node = (Node*)malloc(sizeof(Node));
if (new_node == NULL) {
    printf("Memory allocation failed.\n");
    return;
}

new_patient.id = generate_patient_id();
new_node->data = new_patient;
new_node->next = NULL;

if (head == NULL) {
    head = new_node;
} else {
    Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = new_node;
}

printf("Successfully added patient\n\n");
}

void search_patient() {
    printf("List of Patient Name:\n");
    Node* current = head;
    while (current != NULL) {
        printf("%s\n", current->data.name);
        current = current -> next;
    }

    printf("Enter the patient name who you want to find his/her information: ");
    char name[100];
    scanf("%99s", name);
    current = head;
    int check = 0;
    while (current != NULL) {
        if (strcmp(current->data.name, name) == 0) {
```



```
        printf("Name: %s, Age: %d, Gender: %s\n", current->data.name, current->data.age,
current->data.gender);
        check = 1;
    }
    current = current->next;
}
if (!check) {
    printf("Can not find the given patient name in the list.\n");
}
}

void display_patient_info() {
    Node* current = head;
    printf("Patient List:\n");
    int count = 0;
    while (current != NULL) {
        count++;
        printf("ID: %d, Name: %s, Age: %d, Gender: %s\n", current->data.id, current->data.name,
current->data.age, current->data.gender);
        current = current -> next;
    }

    printf("\n");
}

void delete_patient() {

    printf("List of Patient Name:\n");
    Node* current = head;
    while (current != NULL) {
        printf("%s\n", current->data.name);
        current = current -> next;
    }

    printf("Enter the name that you want to remove in the list:\n");
    char name[50];
    scanf("%s", name);
    current = head;
    Node* prev = NULL;

    while (current != NULL) {
        if (strcmp(current->data.name, name) == 0) {
            if (prev == NULL) {
                head = current->next;
            } else {
                prev->next = current->next;
            }

            printf("Successfully delete that patient\n");
            free(current);
            return;
        }
        prev = current;
        current = current -> next;
    }
}
```



```
}
printf("Patient with name %s not found.\n", name);
}

void free_list() {
    Node* current = head;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    head = NULL;
}

void save_patient_to_file() {
    FILE* f_open = fopen("data/patients.txt", "w");
    Node* current = head;

    if (head == NULL) {
        return;
    }

    while (current != NULL) {
        fprintf(f_open, "%s %d %s\n", current->data.name, current->data.age, current->data.gender);
        current = current -> next;
    }
    fclose(f_open);
}

void load_patient_from_file() {
    FILE* f_open = fopen("data/patients.txt", "r");
    if (!f_open) {
        perror("Error opening patient file");
        return;
    }

    Patient human;
    while (fscanf(f_open, "%49s %d %9s", human.name, &human.age, human.gender) == 3) {
        // Validate gender
        if (!is_valid_gender(human.gender)) {
            printf("Warning: Invalid gender '%s' for patient %s - using 'Unknown'\n",
                human.gender, human.name);
            strcpy(human.gender, "Unknown");
        }

        Node* new_node = (Node*)malloc(sizeof(Node));
        if (!new_node) {
            printf("Memory allocation failed\n");
            break;
        }

        human.id = generate_patient_id();
        new_node->data = human;
        new_node->next = NULL;
    }
}
```



```
if (head == NULL) {
    head = new_node;
} else {
    Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = new_node;
}

printf("Successfully loaded data from patient file!\n");
fclose(f_open);
}
```

doctor.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "doctor.h"
struct Node_doctor *root = NULL;

void init_doctor(Vector* v) {
    v->size = 0;
    v->capacity = 2;
    v->data = malloc(v->capacity * sizeof(Doctor));
}

struct Node_doctor* getNewNode(Doctor doctor) {
    struct Node_doctor *newNode = malloc(sizeof(struct Node_doctor));
    newNode->doctor = doctor;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node_doctor* insert(struct Node_doctor* root, Doctor doctor) {
    if (root == NULL)
        return getNewNode(doctor);
    if (root->doctor.id < doctor.id) {
        root->right = insert(root->right, doctor);
    }
    else if (root->doctor.id > doctor.id) {
        root->left = insert(root->left, doctor);
    }

    return root;
}
```



```
Doctor search(struct Node_doctor* root, int doctor_id) {
    if (root == NULL) {
        printf("Doctor not found.\n");
        exit(1);
    }

    if (root->doctor.id == doctor_id)
        return root->doctor;

    if (root->doctor.id < doctor_id)
        return search(root->right, doctor_id);
    else
        return search(root->left, doctor_id);
}

void search_doctor(Vector* v) {
    printf("This is our doctor id list:\n");
    for (int i = 0; i < v->size; i++) {
        printf("%d ", v->data[i].id);
    }
    printf("\n");
    printf("Enter the id of the doctor that you want to search:\n");
    int doctor_id;
    scanf("%d", &doctor_id);
    Doctor Answer = search(root, doctor_id);
    printf("Name: %s, ID: %d\n", Answer.name, Answer.id);
}

void inorder_doctor(struct Node_doctor* root) {
    if (root == NULL) return;
    inorder_doctor(root->left);
    printf("Name: %s, ID: %d\n", root->doctor.name, root->doctor.id);
    inorder_doctor(root->right);
}

void list_doctor() {
    printf("Doctor List:\n");
    inorder_doctor(root);
    return;
}

void load_doctor_from_file(Vector* v) {
    FILE* f_open = fopen("data/doctors.txt", "r");

    Doctor human;
    while (fscanf(f_open, "%s %d", human.name, &human.id) == 2) {
        if (v->size == v->capacity) {
            v->capacity *= 2;
            v->data = realloc(v->data, v->capacity * sizeof(Doctor));
        }
    }
}
```




```
((Doctor*) (v->data)) [v->size++] = human;

}

for (int i = 0; i < v->size; i++) {
    root = insert(root, v->data[i]);
}

fclose(f_open);
}

void save_doctor_to_file(Vector* v) {
    FILE* f_open = fopen("data/doctors.txt", "w");
    for (int i = 0; i < v->size; i++) {
        fprintf(f_open, "%s %d\n", v->data[i].name, v->data[i].id);
    }
    fclose(f_open);
}

struct Node_doctor* findMin(struct Node_doctor* root) {
    if (root->left == NULL) {
        return root;
    }
    return findMin(root->left);
}

struct Node_doctor* deleteNode(struct Node_doctor* root, int examined_id) {
    if (root == NULL) {
        return NULL;
    }
    if (root->doctor.id > examined_id) root->left = deleteNode(root->left, examined_id);
    if (root->doctor.id < examined_id) root->right = deleteNode(root->right, examined_id);

    if (root->doctor.id == examined_id) {
        if (root->right == NULL && root->left == NULL) {
            free(root);
            return NULL;
        }
        else if (root->right == NULL) {
            struct Node_doctor* temp = root->left;
            free(root);
            return temp;
        }
        else if (root->left == NULL) {
            struct Node_doctor* temp = root->right;
            free(root);
            return temp;
        }
    }
}
```



```
        else {
            struct Node_doctor* Lowest_id_doctor = findMin(root->right);
            root->doctor.id = Lowest_id_doctor->doctor.id;
            root->right = deleteNode(root->right, Lowest_id_doctor->doctor.id);
        }
    }
    return root;
}

void delete_doctor_by_name(Vector* v) {
    printf("Doctor's Name List:\n");
    for (int i = 0 ; i < v->size; i++) {
        printf("%s\n", v->data[i].name);
    }
    int id_result = -1;

    printf("Enter the name of the patient that you want delete:\n");
    char name[50];
    scanf("%s", name);
    int i = 0;
    for (; i < v->size; i++) {
        if (strcmp(v->data[i].name, name) == 0) {
            id_result = v->data[i].id;
            break;
        }
    }

    for (int j = i; j < v->size - 1; j++) {
        v->data[j] = v->data[j + 1];
    }
    v->size--;
    if (id_result == -1) {
        printf("Can not find the given doctor name.\n");
    } else {
        root = deleteNode(root, id_result);
    }
}

void add_doctor(Vector* v) {
    Doctor human;
    printf("Enter the doctor name:\n");
    scanf("%s", human.name);
    printf("Enter the doctor id:\n");
    scanf("%d", &human.id);

    if (v->size == v->capacity) {
        v->capacity *= 2;
        v->data = realloc(v->data, v->capacity * sizeof(Doctor));
    }
}
```



```
v->data[v->size++] = human;
root = insert(root, human);
}

void free_bst_doctor(struct Node_doctor* root) {
    if (root == NULL) return;
    free_bst_doctor(root->left);
    free_bst_doctor(root->right);
    free(root);
}

void free_doctor(Vector *v) {
    free(v->data);
    free_bst_doctor(root);
    root = NULL;
}
```

appointment.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include "appointment.h"
#include "doctor.h"
#include "patient.h"
#include "utils.h"
//BST

void init_appointment_system(AppointmentSystem* system) {
    system->bst_root = NULL;
    system->queue_front = NULL;
    system->queue_rear = NULL;
    system->count = 0;
}

//new node
static BSTNode* new_bst_node(Appointment appt) {
    BSTNode* node = (BSTNode*)malloc(sizeof(BSTNode));
    if (node) {
        node->data = appt;
        node->left = node->right = NULL;
    }
    return node;
}

//process appointment theo thu tu
static QueueNode* new_queue_node(Appointment appt) {
```



```
QueueNode* node = (QueueNode*)malloc(sizeof(QueueNode));
if (node) {
    node->data = appt; //appointment trong node
    node->next = NULL; //pointer tiep theo la NULL
}
return node;
}

void inorder_appointment(struct BSTNode* root) {
    if (root == NULL) {
        return;
    }
    inorder_appointment(root->left);
    printf("Appt #d:\n", root->data.appointment_id);
    printf("Patient information: Name: %s, Age: %d, Gender: %s\n", root->data.patient.name,
root->data.patient.age, root->data.patient.gender);
    printf("Doctor: Name: %s, ID: %d\n\n", root->data.doctor.name, root->data.doctor.id);
    inorder_appointment(root->right);
}

void display_appointment(AppointmentSystem* appt_system) {
    if (appt_system->bst_root == NULL) return;
    inorder_appointment(appt_system->bst_root);
}

int hour = 0, min = 0;
void generate_appointment_from_file(AppointmentSystem* appt_system) {
    FILE* f1 = fopen("data/doctors.txt", "r");
    FILE* f2 = fopen("data/patients.txt", "r");
    FILE* f3 = fopen("data/appointments.txt", "w");

    if (!f1 || !f2 || !f3) {
        perror("Error opening files");
        if (f1) fclose(f1);
        if (f2) fclose(f2);
        if (f3) fclose(f3);
        return;
    }

    Doctor doctor;
    Patient patient;
    time_t current_time = time(NULL);
    struct tm *now = localtime(&current_time);

    int year = now->tm_year + 1900;
    int month = now->tm_mon + 1;
    int day = now->tm_mday;
    int hour = 0; // Start scheduling from 00:00
    int min = 0;

    printf("\nCurrent Date: %02d/%02d/%d\n\n", day, month, year);
```



```
while ((fscanf(f1, "%99s %d", doctor.name, &doctor.id) == 2) &&
      (fscanf(f2, "%99s %d %9s", patient.name, &patient.age, patient.gender) == 3)) {

    struct tm appt_tm = *now;
    appt_tm.tm_hour = hour;
    appt_tm.tm_min = min;
    appt_tm.tm_sec = 0;
    time_t appt_time = mktime(&appt_tm);

    struct tm *normalized = localtime(&appt_time);

    printf("Doctor: %-15s ID: %-4d | Patient: %-15s Age: %-3d Gender: %-6s\n",
           doctor.name, doctor.id, patient.name, patient.age, patient.gender);

    printf("Appointment Time: %02d:%02d on %02d/%02d/%d - ",
           normalized->tm_hour, normalized->tm_min,
           normalized->tm_mday, normalized->tm_mon + 1,
           normalized->tm_year + 1900);

    if (book_appointment(appt_system, patient, doctor, appt_time)) {
        printf("Booked successfully!\n\n");
        fprintf(f3, "DR: %s (ID:%d) | PATIENT: %s | TIME: %02d:%02d %02d/%02d/%d\n",
                doctor.name, doctor.id, patient.name,
                normalized->tm_hour, normalized->tm_min,
                normalized->tm_mday, normalized->tm_mon + 1,
                normalized->tm_year + 1900);
    } else {
        printf("Failed - time slot occupied\n\n");
    }

    hour++;
    if (hour >= 24) {
        hour = 0;
        day++;
        appt_tm = *now;
        appt_tm.tm_mday = day;
        mktime(&appt_tm); // Normalize date
    }
}

while (fscanf(f1, "%99s %d", doctor.name, &doctor.id) == 2) {
    printf("Unmatched Doctor: %s (ID: %d)\n", doctor.name, doctor.id);
}

while (fscanf(f2, "%99s %d %9s", patient.name, &patient.age, patient.gender) == 3) {
    printf("Unmatched Patient: %s (Age: %d)\n", patient.name, patient.age);
}

fclose(f1);
fclose(f2);
fclose(f3);
}
```



```
//book appointment
int book_appointment(AppointmentSystem* system, Patient patient, Doctor doctor, time_t datetime) {
    Appointment appt;
    appt.appointment_id = system->count + 1; //tao ID cho appointment
    appt.patient = patient;
    appt.doctor = doctor;
    appt.datetime = datetime; //store datetime appointment

    BSTNode** current = &system->bst_root;
    while (*current != NULL) {
        if (datetime < (*current)->data.datetime) {
            current = &((*current)->left);
        } else if (datetime > (*current)->data.datetime) {
            current = &((*current)->right);
        } else {
            return 0;
        }
    }

    *current = new_bst_node(appt); // Gắn node mới vào cây
    if (*current == NULL) return 0;

    //queue node cho appointment
    QueueNode* q_node = new_queue_node(appt);
    if (q_node == NULL) return 0;

    if (system->queue_rear == NULL) {
        system->queue_front = system->queue_rear = q_node;
    } else {
        system->queue_rear->next = q_node;
        system->queue_rear = q_node;
    }

    system->count++;
    return 1;
}

// Hiển thị các phần tử trong queue
void display_queue_appointment(AppointmentSystem* appt_system) {
    if (appt_system->queue_front == NULL) {
        printf("There are no patient in the current list\n");
        return;
    }
    QueueNode* head = appt_system->queue_front;
    while (head != NULL) {
        Appointment data = head->data;
        printf("Appt #d:\n", data.appointment_id);
        printf("Patient information: Name: %s, Age: %d, Gender: %s\n", data.patient.name, data.patient.age,
data.patient.gender);
        printf("Doctor: Name: %s, ID: %d\n\n", data.doctor.name, data.doctor.id);
        printf("Time: %s\n\n", ctime(&data.datetime));
        head = head->next;
    }
}
```



```
}

printf("\n");
}

void pick_next_appointment(AppointmentSystem* appt_system) {
    if (appt_system->queue_front == NULL) return;
    QueueNode* next = appt_system->queue_front;
    Appointment front_data = next->data;

    printf("\nNext Appointment:\n");
    printf("Name: %s, Age: %d, Gender: %s\n", front_data.patient.name, front_data.patient.age,
front_data.patient.gender);
    printf("Name: %s, ID: %d\n", front_data.doctor.name, front_data.doctor.id);
    printf("Time: %s", ctime(&front_data.datetime));

    appt_system->queue_front = appt_system->queue_front->next;

    if (appt_system->queue_front == NULL) {
        appt_system->queue_rear = NULL;
    }

    free(next);
}

//free memory
void free_bst(BSTNode* node) {
    if (node) {
        free_bst(node->left);
        free_bst(node->right);
        free(node);
    }
}

void free_appointment_system(AppointmentSystem* system) {
    free_bst(system->bst_root);

    while (system->queue_front) {
        QueueNode* temp = system->queue_front;
        system->queue_front = system->queue_front->next;
        free(temp);
    }
    free(system->queue_front);
    free(system->queue_rear = NULL);
    system->count = 0;
}
```

utils.c



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "utils.h"
#include <time.h>

time_t create_time(int year, int mon, int day, int hour, int min) {
    struct tm tm = {0};
    tm.tm_year = year - 1900;
    tm.tm_mon = mon - 1;
    tm.tm_mday = day;
    tm.tm_hour = hour;
    tm.tm_min = min;
    return mktime(&tm);
}

int count_from_one = 0;

int generate_patient_id() {
    count_from_one++;
    return count_from_one;
}

int is_valid_gender(const char* gender) {
    return (strcasecmp(gender, "Male") == 0) ||
           (strcasecmp(gender, "Female") == 0);
}
```

patient.h

```
#ifndef PATIENT_H
#define PATIENT_H

#define MAX_NAME_LEN 49
#define MAX_GENDER_LEN 9

typedef struct {
    int id;
    char name[MAX_NAME_LEN + 1];
    int age;
    char gender[MAX_GENDER_LEN + 1];
} Patient;
```




```
typedef struct Node {
    Patient data;
    struct Node* next;
} Node;

void add_patient();
void save_patient_to_file();
void load_patient_from_file();
void search_patient();
void display_patient_info();
void delete_patient();
void free_list();

#endif
```

doctor.h

```
#ifndef DOCTOR_H
#define DOCTOR_H

// Cấu trúc Doctor
typedef struct {
    char name[100];
    int id;
} Doctor;

struct Node_doctor {
    Doctor doctor;
    struct Node_doctor* left;
    struct Node_doctor* right;
};

typedef struct {
    Doctor* data;
    int size;
    int capacity;
} Vector;

void delete_doctor_by_name(Vector* v);
void init_doctor(Vector* v);
void load_doctor_from_file(Vector* v);
void list_doctor();
void add_doctor(Vector* v);
void search_doctor(Vector* v);
void save_doctor_to_file(Vector* v);
void delete_doctor_by_name(Vector* v);
struct Node_doctor* getNewNode(Doctor doctor);
struct Node_doctor* insert(struct Node_doctor* root, Doctor doctor);
Doctor search(struct Node_doctor* root, int doctor_id);
void search_doctor(Vector* v);
void inorder_doctor(struct Node_doctor* root);
void free_doctor(Vector* v);
```



```
#endif // DOCTOR_H
```

appointment.h

```
#ifndef APPOINTMENT_H
#define APPOINTMENT_H

#include "patient.h"
#include "doctor.h"
#include <time.h>

#define MAX_PURPOSE_LEN 100

typedef struct {
    int appointment_id;
    Patient patient;
    Doctor doctor;
    time_t datetime;
} Appointment;

// BST thoi gian
typedef struct BSTNode {
    Appointment data;
    struct BSTNode* left;
    struct BSTNode* right;
} BSTNode;

// Queue process order bệnh nhân
typedef struct QueueNode {
    Appointment data;
    struct QueueNode* next;
} QueueNode;

typedef struct {
    BSTNode* bst_root;
    QueueNode* queue_front;
    QueueNode* queue_rear;
    int count;
} AppointmentSystem;

void init_appointment_system(AppointmentSystem* system);
int book_appointment(AppointmentSystem* system, Patient patient, Doctor doctor, time_t datetime);
Appointment* get_next_appointment(const AppointmentSystem* system);
void find_appointments_in_range(const AppointmentSystem* system,
    time_t start,
    time_t end,
    const Appointment** results, // Keep const here
    int* count);

void free_appointment_system(AppointmentSystem* system);
void display_appointment(AppointmentSystem* system);
```



```
void free_appointment_system(AppointmentSystem* system);  
void pick_next_appointment(AppointmentSystem* appt_system);  
void inorder_appointment(struct BSTNode* root);  
void generate_appointment_from_file(AppointmentSystem* appt_system);  
void free_bst(BSTNode* node);  
void display_queue_appointment(AppointmentSystem* system);  
  
#endif
```

utils.h

```
#ifndef UTILS_H  
#define UTILS_H  
  
#include <time.h>  
#include <stdbool.h> // For bool type  
  
/* Time Utility */  
time_t create_time(int year, int mon, int day, int hour, int min);  
/* ID Generation */  
int generate_patient_id(void);  
//gender  
int is_valid_gender(const char* gender);  
  
#endif // UTILS_H
```



4. TESTING:

4.1 Testing for functions related to Patients:

Start:

```
PS D:\Kien Hoang\Kien Hoang\Management_Hospitality> .\hospital_system.exe

=== Hospital Management System ===

Patient Management:
1. Load patients from file
2. Add a patient
3. Search a patient
4. Delete a patient
5. Display patient information
6. Save patients to file

Doctor Management:
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: █
```

Here is the patient list we already have in patients.txt:

```
Management_Hospitality > data > patients.txt
1 Alice 30 Female
2 Bob 25 Male
3 Charlie 40 Male
4 Diana 35 Female
5 Ethan 28 Male
6 Fiona 32 Female
7 George 45 Male
8 Hannah 29 Female
9 Ivan 38 Male
10 Long 98 Male
11 Salah 25 Male
12 Kante 28 Male
13 Ahri 29 Female
14 Hanh 18 Female
15 Raz 27 Female
16 Xiao 25 Male
17 Thanh 22 Male
18 Longdeptrao 21 Male
19 Minh 30 Female
20 Linh 22 Female
21 Ultron 21 Male
22 Shelby 22 Male
23 Ekko 21 Male
24 Jinx 42 Female
25 Lenna 30 Female
```



When we load it (Choice 1), we have this:

```
=== Hospital Management System ===

Patient Management:
1. Load patients from file
2. Add a patient
3. Search a patient
4. Delete a patient
5. Display patient information
6. Save patients to file

Doctor Management:
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 1
Successfully loaded data from patient file!
```

Now when we add a patient (Choice 2):

```
Patient Management:
1. Load patients from file
2. Add a patient
3. Search a patient
4. Delete a patient
5. Display patient information
6. Save patients to file

Doctor Management:
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 2
Enter the patient's name: Ronaldo
Enter the patient's age: 40
Enter the patient's gender (Male/Female): Male
Successfully added patient
```



We add again but with invalid gender, age, it will respawn the command:

```
0. Exit
Enter your choice: 2
Enter the patient's name: Ronaldo
Enter the patient's age: robot
Invalid age. Please enter a positive number: 22
Enter the patient's gender (Male/Female): robot
Invalid gender. Please enter either 'Male' or 'Female'.
Enter the patient's gender (Male/Female): Female
Successfully added patient
```

Now we search a patient (Choice 3) given 2 identical name patients:

```
Enter your choice: 3
List of Patient Name:
Alice
Bob
Charlie
Diana
Ethan
Fiona
George
Hannah
Ivan
Long
Salah
Kante
Ahri
Hanh
Raz
Xiao
Thanh
Longdeptraai
Minh
Linh
Ultron
Shelby
Ekko
Jinx
Loran
Ronaldo
Ronaldo
Enter the patient name who you want to find his/her information: Ronaldo
Name: Ronaldo, Age: 40, Gender: Male
Name: Ronaldo, Age: 22, Gender: Female
```



Delete a patient (choice 4) - one weakness we should improve in the future is that it will delete the patient who was added first if we have 2 patients with identical names (as we can see in choice 5 after this image):

```
0. Exit
Enter your choice: 4
List of Patient Name:
Alice
Bob
Charlie
Diana
Ethan
Fiona
George
Hannah
Ivan
Long
Salah
Kante
Ahri
Hanh
Raz
Xiao
Thanh
Longdeptraï
Minh
Linh
Ultron
Shelby
Ekko
Jinx
Loran
Ronaldo
Ronaldo
Enter the name that you want to remove in the list:
Ronaldo
Successfully delete that patient
```



Display patient information (Choice 5):

```
Enter your choice: 5
Patient List:
ID: 1, Name: Alice, Age: 30, Gender: Female
ID: 2, Name: Bob, Age: 25, Gender: Male
ID: 3, Name: Charlie, Age: 40, Gender: Male
ID: 4, Name: Diana, Age: 35, Gender: Female
ID: 5, Name: Ethan, Age: 28, Gender: Male
ID: 6, Name: Fiona, Age: 32, Gender: Female
ID: 7, Name: George, Age: 45, Gender: Male
ID: 8, Name: Hannah, Age: 29, Gender: Female
ID: 9, Name: Ivan, Age: 38, Gender: Male
ID: 10, Name: Long, Age: 98, Gender: Male
ID: 11, Name: Salah, Age: 25, Gender: Male
ID: 12, Name: Kante, Age: 28, Gender: Male
ID: 13, Name: Ahri, Age: 29, Gender: Female
ID: 14, Name: Hanh, Age: 18, Gender: Female
ID: 15, Name: Raz, Age: 27, Gender: Female
ID: 16, Name: Xiao, Age: 25, Gender: Male
ID: 17, Name: Thanh, Age: 22, Gender: Male
ID: 18, Name: Longdeptra, Age: 21, Gender: Male
ID: 19, Name: Minh, Age: 30, Gender: Female
ID: 20, Name: Linh, Age: 22, Gender: Female
ID: 21, Name: Ultron, Age: 21, Gender: Male
ID: 22, Name: Shelby, Age: 22, Gender: Male
ID: 23, Name: Ekko, Age: 21, Gender: Male
ID: 24, Name: Jinx, Age: 42, Gender: Female
ID: 25, Name: Loran, Age: 20, Gender: Female
ID: 27, Name: Ronaldo, Age: 22, Gender: Female
```

Finally, we saved patients to file (Choice 6):

```
Patient Management:
1. Load patients from file
2. Add a patient
3. Search a patient
4. Delete a patient
5. Display patient information
6. Save patients to file

Doctor Management:
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 6
```




Cases in choice 3 and choice 4 (Find, delete a patient), but the name doesn't exist:

```
Ekko
Jinx
Loran
Ronaldo
Enter the name that you want to remove in the list:
namaste
Patient with name namaste not found.

=== Hospital Management System ===
```

```
Shelby
Ekko
Jinx
Loran
Ronaldo
Enter the patient name who you want to find his/her information: Rasputin
Can not find the given patient name in the list.

=== Hospital Management System ===
```

4.2 Testing for functions related to Doctors:

Similarly, when we start with choice 7 (load doctors from a file) it will load the list of doctors from doctor.txt to the system:

```
Management_Hospitality > data > doctors.txt
1 Alex 101
2 Sophia 102
3 Liam 103
4 Olivia 104
5 Noah 105
6 Emma 106
7 William 107
8 Ava 108
9 James 109
10 Isabella 110
11 Benjamin 111
12 Mia 112
13 Elijah 113
14 Charlotte 114
15 Henry 115
16 Amelia 116
17 Lucas 117
18 Evelyn 118
19 Jackson 119
20 Harper 120
21 Daniel 121
22 Lily 122
23 Michael 123
24 Ella 124
25 Sebastian 125
26 Katarina 200
27
```

from this list to the list doctor (choice 10)



```
Enter your choice: 10
Doctor List:
Name: Alex, ID: 101
Name: Sophia, ID: 102
Name: Liam, ID: 103
Name: Olivia, ID: 104
Name: Noah, ID: 105
Name: Emma, ID: 106
Name: William, ID: 107
Name: Ava, ID: 108
Name: James, ID: 109
Name: Isabella, ID: 110
Name: Benjamin, ID: 111
Name: Mia, ID: 112
Name: Elijah, ID: 113
Name: Charlotte, ID: 114
Name: Henry, ID: 115
Name: Amelia, ID: 116
Name: Lucas, ID: 117
Name: Evelyn, ID: 118
Name: Jackson, ID: 119
Name: Harper, ID: 120
Name: Daniel, ID: 121
Name: Lily, ID: 122
Name: Michael, ID: 123
Name: Ella, ID: 124
Name: Sebastian, ID: 125
Name: Katarina, ID: 200
```

However, if we don't load the list and pick choice 10 (show list doctor), it won't show:

```
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 10
Doctor List:

=== Hospital Management System ===

Patient Management:
```



Now we choose choice 8 (add doctor):

```
Doctor Management:
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 8
Enter the doctor name:
Nani
Enter the doctor id:
247

=== Hospital Management System ===
```

And the added doctor will be shown to the list (the list below isn't loaded from the doctor.txt, so we only have 1 name):

```
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 10
Doctor List:
Name: Nani, ID: 247

=== Hospital Management System ===
```



However, if we load doctor (choice 7) again it will show all the names including the newest added one:

```
0. Exit
Enter your choice: 10
Doctor List:
Name: Alex, ID: 101
Name: Sophia, ID: 102
Name: Liam, ID: 103
Name: Olivia, ID: 104
Name: Noah, ID: 105
Name: Emma, ID: 106
Name: William, ID: 107
Name: Ava, ID: 108
Name: James, ID: 109
Name: Isabella, ID: 110
Name: Benjamin, ID: 111
Name: Mia, ID: 112
Name: Elijah, ID: 113
Name: Charlotte, ID: 114
Name: Henry, ID: 115
Name: Amelia, ID: 116
Name: Lucas, ID: 117
Name: Evelyn, ID: 118
Name: Jackson, ID: 119
Name: Harper, ID: 120
Name: Daniel, ID: 121
Name: Lily, ID: 122
Name: Michael, ID: 123
Name: Ella, ID: 124
Name: Sebastian, ID: 125
Name: Katarina, ID: 200
Name: Nani, ID: 247
```

Delete the doctor:

```
Michael
Ella
Sebastian
Katarina
Enter the name of the patient that you want delete:
Nani

=== Hospital Management System ===
```



Check the list (choice 10) again to ensure the name is deleted:

```
0. Exit
Enter your choice: 10
Doctor List:
Name: Alex, ID: 101
Name: Sophia, ID: 102
Name: Liam, ID: 103
Name: Olivia, ID: 104
Name: Noah, ID: 105
Name: Emma, ID: 106
Name: William, ID: 107
Name: Ava, ID: 108
Name: James, ID: 109
Name: Isabella, ID: 110
Name: Benjamin, ID: 111
Name: Mia, ID: 112
Name: Elijah, ID: 113
Name: Charlotte, ID: 114
Name: Henry, ID: 115
Name: Amelia, ID: 116
Name: Lucas, ID: 117
Name: Evelyn, ID: 118
Name: Jackson, ID: 119
Name: Harper, ID: 120
Name: Daniel, ID: 121
Name: Lily, ID: 122
Name: Michael, ID: 123
Name: Ella, ID: 124
Name: Sebastian, ID: 125
Name: Katarina, ID: 200

=== Hospital Management System ===
```

Similarly, with the Patient module, we can't delete an unreal name:

```
Katarina
Enter the name of the patient that you want delete:
LeTheDoan
Can not find the given doctor name.

=== Hospital Management System ===
```

Pick choice 11 to save:

```
7. Load doctors from file
8. Add doctor
9. Delete a doctor
10. List Doctors
11. Save doctors to file

Appointment:
12. Generate appointment from doctor and patient
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 11

=== Hospital Management System ===
```



4.3 Testing for Appointment

For appointment, we pick choice 12 to generate appointments accordingly for all patients and doctors (we are using time from 0:00 to 23:00 for the demo purpose only, not for practical purpose). Thus we were having troubles with updating date (day, month, year) when it goes over 24:00. So we decided it would be a weakness to work on in the future:

Current Date: 07/04/2025

Doctor: Alex ID: 101 | Patient: Alice Age: 30 Gender: Female
Appointment Time: 00:00 on 07/04/2025 - Booked successfully!

Doctor: Sophia ID: 102 | Patient: Bob Age: 25 Gender: Male
Appointment Time: 01:00 on 07/04/2025 - Booked successfully!

Doctor: Liam ID: 103 | Patient: Charlie Age: 40 Gender: Male
Appointment Time: 02:00 on 07/04/2025 - Booked successfully!

Doctor: Olivia ID: 104 | Patient: Diana Age: 35 Gender: Female
Appointment Time: 03:00 on 07/04/2025 - Booked successfully!

Doctor: Noah ID: 105 | Patient: Ethan Age: 28 Gender: Male
Appointment Time: 04:00 on 07/04/2025 - Booked successfully!

Doctor: Emma ID: 106 | Patient: Fiona Age: 32 Gender: Female
Appointment Time: 05:00 on 07/04/2025 - Booked successfully!

Doctor: William ID: 107 | Patient: George Age: 45 Gender: Male
Appointment Time: 06:00 on 07/04/2025 - Booked successfully!

Doctor: Ava ID: 108 | Patient: Hannah Age: 29 Gender: Female
Appointment Time: 07:00 on 07/04/2025 - Booked successfully!

Doctor: James ID: 109 | Patient: Ivan Age: 38 Gender: Male
Appointment Time: 08:00 on 07/04/2025 - Booked successfully!

Doctor: Isabella ID: 110 | Patient: Long Age: 98 Gender: Male
Appointment Time: 09:00 on 07/04/2025 - Booked successfully!

Doctor: Benjamin ID: 111 | Patient: Salah Age: 25 Gender: Male
Appointment Time: 10:00 on 07/04/2025 - Booked successfully!

Doctor: Mia ID: 112 | Patient: Kante Age: 28 Gender: Male
Appointment Time: 11:00 on 07/04/2025 - Booked successfully!

Doctor: Elijah ID: 113 | Patient: Ahri Age: 29 Gender: Female
Appointment Time: 12:00 on 07/04/2025 - Booked successfully!



```
Doctor: Elijah      ID: 113 | Patient: Ahri      Age: 29  Gender: Female
Appointment Time: 12:00 on 07/04/2025 - Booked successfully!

Doctor: Charlotte   ID: 114 | Patient: Hanh      Age: 18  Gender: Female
Appointment Time: 13:00 on 07/04/2025 - Booked successfully!

Doctor: Henry       ID: 115 | Patient: Raz      Age: 27  Gender: Female
Appointment Time: 14:00 on 07/04/2025 - Booked successfully!

Doctor: Amelia      ID: 116 | Patient: Xiao      Age: 25  Gender: Male
Appointment Time: 15:00 on 07/04/2025 - Booked successfully!

Doctor: Lucas       ID: 117 | Patient: Thanh     Age: 22  Gender: Male
Appointment Time: 16:00 on 07/04/2025 - Booked successfully!

Doctor: Evelyn      ID: 118 | Patient: Longdeptraí Age: 21  Gender: Male
Appointment Time: 17:00 on 07/04/2025 - Booked successfully!

Doctor: Jackson     ID: 119 | Patient: Minh     Age: 30  Gender: Female
Appointment Time: 18:00 on 07/04/2025 - Booked successfully!

Doctor: Harper      ID: 120 | Patient: Linh     Age: 22  Gender: Female
Appointment Time: 19:00 on 07/04/2025 - Booked successfully!

Doctor: Daniel      ID: 121 | Patient: Ultron    Age: 21  Gender: Male
Appointment Time: 20:00 on 07/04/2025 - Booked successfully!

Doctor: Lily        ID: 122 | Patient: Shelby    Age: 22  Gender: Male
Appointment Time: 21:00 on 07/04/2025 - Booked successfully!

Doctor: Michael     ID: 123 | Patient: Ekko     Age: 21  Gender: Male
Appointment Time: 22:00 on 07/04/2025 - Booked successfully!

Doctor: Ella        ID: 124 | Patient: Jinx     Age: 42  Gender: Female
Appointment Time: 23:00 on 07/04/2025 - Booked successfully!

Doctor: Sebastian   ID: 125 | Patient: Loran    Age: 20  Gender: Female
Appointment Time: 00:00 on 07/04/2025 - Failed - time slot occupied

Unmatched Patient: Ronaldo (Age: 22)

=== Hospital Management System ===
```

Choice 13 will show original appointments as below:

```
Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 13
Appt #1:
Patient information: Name: Alice, Age: 30, Gender: Female
Doctor: Name: Alex, ID: 101

Appt #2:
Patient information: Name: Bob, Age: 25, Gender: Male
Doctor: Name: Sophia, ID: 102

Appt #3:
Patient information: Name: Charlie, Age: 40, Gender: Male
Doctor: Name: Liam, ID: 103

Appt #4:
Patient information: Name: Diana, Age: 35, Gender: Female
Doctor: Name: Olivia, ID: 104

Appt #5:
Patient information: Name: Ethan, Age: 28, Gender: Male
Doctor: Name: Noah, ID: 105

Appt #6:
Patient information: Name: Fiona, Age: 32, Gender: Female
Doctor: Name: Emma, ID: 106

Appt #7:
Patient information: Name: George, Age: 45, Gender: Male
Doctor: Name: William, ID: 107
```



Choice 14 (queue appointment will show next appointments that are queueing):

```
Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 14
Appt #1:
Patient information: Name: Alice, Age: 30, Gender: Female
Doctor: Name: Alex, ID: 101

Time: Mon Apr 07 00:00:00 2025

Appt #2:
Patient information: Name: Bob, Age: 25, Gender: Male
Doctor: Name: Sophia, ID: 102

Time: Mon Apr 07 01:00:00 2025

Appt #3:
Patient information: Name: Charlie, Age: 40, Gender: Male
Doctor: Name: Liam, ID: 103

Time: Mon Apr 07 02:00:00 2025

Appt #4:
Patient information: Name: Diana, Age: 35, Gender: Female
Doctor: Name: Olivia, ID: 104

Time: Mon Apr 07 03:00:00 2025

Appt #5:
Patient information: Name: Ethan, Age: 28, Gender: Male
Doctor: Name: Noah, ID: 105
```

Choice 15 will pick the next appointment, meaning it will consider the past appointment is done:

```
Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 15

Next Appointment:
Name: Alice, Age: 30, Gender: Female
Name: Alex, ID: 101
Time: Mon Apr 07 00:00:00 2025

=== Hospital Management System ===
```

if we pick 15 again, this appointment in 0:00:00 will be considered as done



```
Appointment:
12. Generate appointment from doctor and patient
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 15

Next Appointment:
Name: Bob, Age: 25, Gender: Male
Name: Sophia, ID: 102
Time: Mon Apr 07 01:00:00 2025

=== Hospital Management System ===
```

like this

But if we pick choice 13, the original appointment including the done ones will be shown:

```
Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 13
Appt #1:
Patient information: Name: Alice, Age: 30, Gender: Female
Doctor: Name: Alex, ID: 101

Appt #2:
Patient information: Name: Bob, Age: 25, Gender: Male
```

But the queue one only shown next ones:

```
Appointment:
12. Generate appointment from doctor and patient file
13. Display original appointments
14. Display the queue appointment
15. Pick the next appointment to treat

0. Exit
Enter your choice: 14
Appt #3:
Patient information: Name: Charlie, Age: 40, Gender: Male
Doctor: Name: Liam, ID: 103

Time: Mon Apr 07 02:00:00 2025

Appt #4:
```



5. CONCLUSION AND FUTURE WORK:

Throughout the development of MediTrack, a hospital management system written in C, we implemented core functionalities to manage patients, doctors, and appointments in a structured and modular way. The system utilizes linked lists for patient management, a dynamic array and BST for doctor data, and a combination of BST and queue structures for appointment scheduling. We applied file I/O to ensure persistent data storage between sessions.

Strengths of our system include:

- Modular design with separate header and source files for each functionality.
- Use of appropriate data structures to simulate real-world scenarios.
- Fully functional file I/O operations for loading and saving patient and doctor data.
- A working appointment system with queue-based handling and BST-based time lookup.
- Real-time user input and feedback on operations (e.g., adding, searching, deleting).

However, we also acknowledge some limitations discovered during testing and we consider them as future work we can consider in the future to complete the project:

- If multiple patients have the same name, deleting a patient always removes the first match found in the list, with no prompt to choose between duplicates.
- Searching and deleting doctors or patients rely solely on exact string matches, which is not always user-friendly.
- The appointment time system operates on a 24-hour clock without supporting changes in date, month, or year. This limits long-term appointment planning.
- The current interface is purely command-line-based, which may not be accessible for all user types.



6. TABLE OF CONTENTS

INTRODUCTION.....	1
Member List & Workload.....	1
Overview of Product	2
PROBLEM.....	3
Objectives/Problems.....	4
Context and Features	4
SOLUTION	5
Overall Flowchart	5
Modules Architecture	6
Algorithms of All Important Functions	9
Implementation	17
TESTING	34
CONCLUSION AND FUTURE WORK	48
TABLE OF CONTENTS.....	49
REFERENCE LIST.....	50
PRESENTATION VIDEO LINK.....	50



7. REFERENCE LIST.

1. Google. "Google C++ Style Guide." *Google.github.io*,
google.github.io/styleguide/cppguide.html.
2. "Hospital Management System in C." *GeeksforGeeks*, 25 Sept. 2023,
www.geeksforgeeks.org/c-program-for-hospital-management-system/.
3. Kernighan, Brian, and Dennis Ritchie. *The c Programming Language*. 1988.
4. "Top 25 c Projects with Source Code in 2023." *GeeksforGeeks*, 22 Aug. 2023,
www.geeksforgeeks.org/c-projects/.

Presentation Video Link:

https://drive.google.com/file/d/1MA_asiPAbHxaO63YT_3qoYj5acXgbw8J/view?usp=sharing



Assessment rubrics: Your grade is based on the expected delivery column and detailed grading evaluation below.

Rubric for Report and program.

Criteria	Expected delivery	Grade (%)
Overall Report presentation	• A clear and concise explanation the working principle of the program (2.5 %) • The report flows the guideline (format, content,...) (2.5 %)	5
Flow chart	• Cover all the program (2.5%) • A clear and concise explanation the principle of your product. (2.5%)	5
Modules	• Draw the modules including inputs, outputs, ...	5
Pseudo code	• A clear declaration the pseudo code of algorithms	5
Module with headers file (.h; .c)	• Divide clearly your program into modules (headers file *.h, *.c), create the Makefile.	5
Using I/O file	• Using files to store data, read, write the data (2.5%) • Change the information in files (2.5%)	5
At least 5 functions	• Have at least 5 functions	5
Minimum of 300 lines of code	• Minimum of 300 lines of code	5
Complexity of the project	• Good program: creative, new novelty	5



Implementation	• Show the module, code block clearly, style must follow: Google C++ Style Guide	5
Testing	• Have the test cases correct with the functions. • Show the pictures for the testing results, discuss the testing results	10
Conclusion	• The conclusion relates to the program with the guideline	5
Total		65



Rubric for Video record and Presentation:

Your team should record a video (maximum 20 minutes) to present the project.

Criteria	Description	Grade (%)
Targets of Project	Clear explanation of the project's objectives and goals.	3
Functions	Demonstration of key functions implemented in the program.	3
Flow Chart	Presentation of a clear and comprehensive flowchart depicting the program's logic and structure.	3
Modules	Explanation of how the program is modularized, including the identification of different modules and their functionalities.	3
Pseudo Code	Clear presentation of pseudo code illustrating the algorithms used in the program.	3
Header Files (.h, .c)	Explanation of the header files (.h) and source code files (.c) used in the project, including their roles and organization.	3
Using I/O Files	Demonstration of reading from and writing to files as part of the program's input/output operations.	3
Complexity of the Project	Discussion on the complexity of the project, including any advanced features, algorithms, or data structures utilized.	3
Implementation	Evaluation of the effectiveness and efficiency of the program's implementation, considering factors such as code readability, optimization, and error handling.	3
Testing	Presentation of the testing process (run your code), including the types of tests conducted (e.g., unit tests, integration tests) and the results obtained.	6
Conclusion	Summary of key findings, challenges encountered, lessons learned, and future improvements for the project.	2
	TOTAL	35

